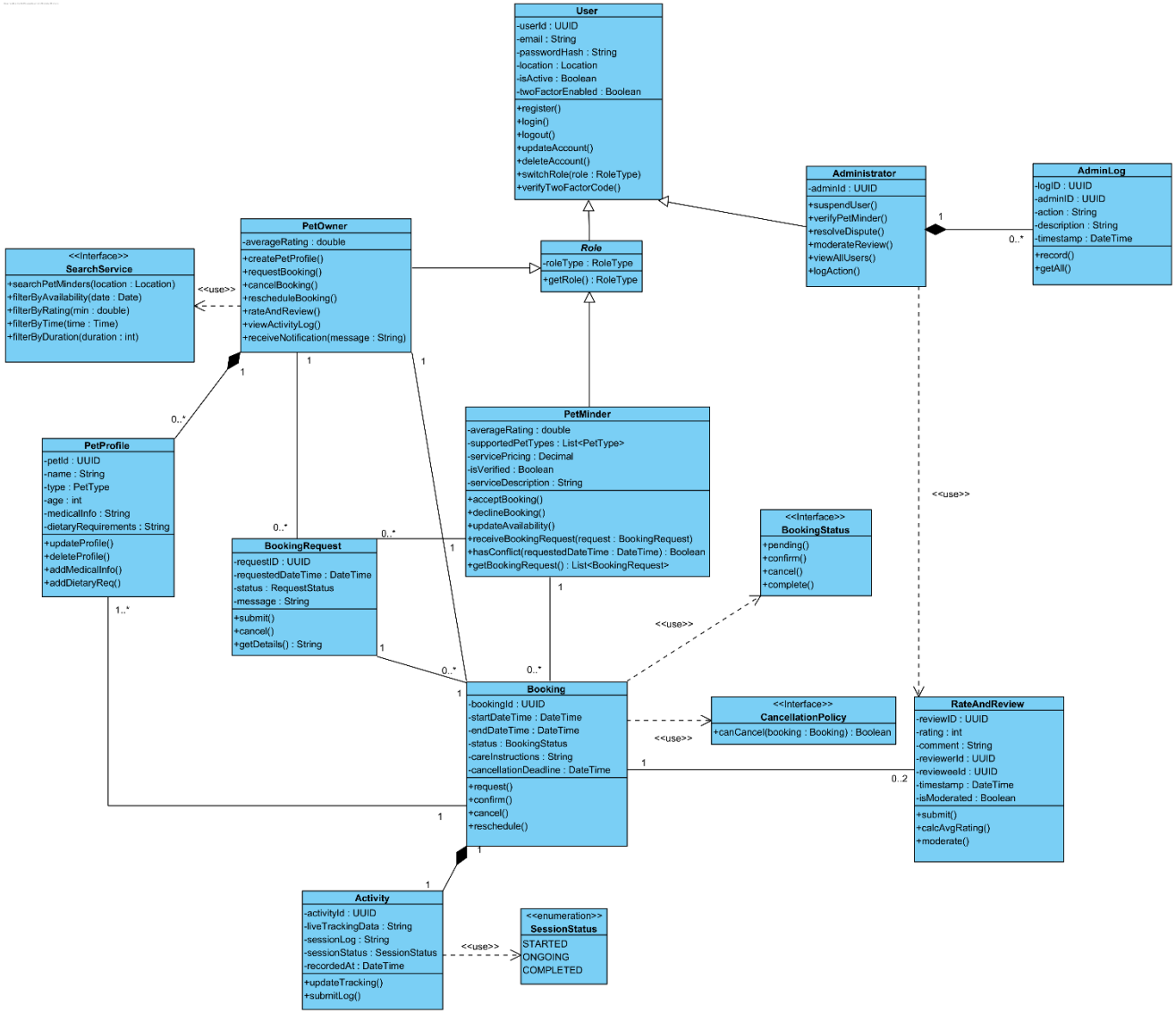
Group 18

Pet Minder

ECS506U Software Engineering Group Project

Design Report

1. Class Diagram (40%)



2. Traceability matrix (10%)

In the traceability matrix map, all the data requirements you identified to a class from your class diagram.

In the explanation, briefly explain how the requirement is fulfilled by the class design.

Class	Requirement	Brief Explanation
User	URF 1.1	The role concept is portrayed through the Role class and switchRole(role) method which allows users to operate as PetOwner and PetMinder
User	URF 1.2	The email attribute in the User class stores the users email address required to registration ad identification
User	URF 1.3	The passwordHash attribute in the User class stores the users hashed password in a secure manner.
User	URF 1.4	The twoFactorEnabled attribute together with the verifyTwoFactorCode() method in the User class supports two-factor authentication during the login process.
User	URF 1.5	The login() method of the User class allows the users to acess their accounts.
User	URF 1.6	The logout() method of the User class allows the user to safely end their session.
User	URF 1.7	The updateAccount() method of the User class allows the user to safely update their account information.
User	URF 1.8	The deleteAccount() method of the user class allows the user to delete their account.
User	URF 1.9	The location attribute of the User class allows the users general location to be stored for matching purposes between pet owners and pet minders.
Role	URF 1.10	The Role class allows role-based system access control between PetOwner and PetMinder users.
User	URF 1.11	The switchRole(role) method in the User class allows a user to switch between PetOwner and PetMinder roles within the system
RateAndReview	URF 1.12	The rating attribute of the RateAndReview class stores the rating given by a pet owner
RateAndReview	URF 1.13	The comment attribute of the RateAndReview class stores the written review.
RateAndReview	URF 1.14	The calcAvgRating() method of the RateAndReview class caluclates the average rating for pet minders.
PetOwner	URF 2.1	The association between PetOwner and PetProfile allows owners to maintain multiple pets.
PetProfile	URF 2.2	The name attribute of the PetProfile class stores the name of each pet.
PetProfile	URF 2.3	The type attribute of the PetProfile class stores the type of pet.
PetProfile	URF 2.4	The age attribute of the PetProfile class stores the age of the pet.

PetProfile	URF 2.5	The medicalInfo attribute of the PetProfile class stores the medical details of the pet.
PetProfile	URF 2.6	The dietaryRequirements attribute of the PetProfile class stores the dietary requirements of the pet.
PetProfile	URF 2.7	The updateProfile() method of the PetProfile class allows the owner to edit an existing pet profile.
PetProfile	URF 2.8	The deleteProfile() method of the PetProfile class allows the owner to delete an eixisting pet profile.
PetMinder	URF 2.9	The location attribute inherited from the User class allows the SearchService.searchPetMinders(location) method to filter Pet Minders based on geographical location.
PetMinder	URF 2.10	The updateAvailability() method of the PetMinder class allows the PetOwner to filter search results by availability.
PetMinder	URF 2.11	The averageRating attribute of the PetMinder class stores the calculated ratings from reviews.
PetMinder	URF 2.12	The SearchService interface and its searchPetMinders() method retrieve and display a list of matching PetMinder objects based on the search criteria offered by the PetOwner.
PetMinder	URF 2.13	The qualifications, experience, serviceDescription, and averageRating attributes in the PetMinder class provide the information required to display a complete Pet Minder profile to a Pet Owner.
Booking	URF 2.14	The request() method of the Booking class creates a booking request between pet owner and pet minder.
Booking	URF 2.15	The startDateTime and endDateTime attributes of the Booking class store the booking duration.
Booking	URF 2.16	The careInstructions attribute of the Booking class store the instructions given by the pet owner.
Booking	URF 2.17	The cancel() method of the Booking class allows bookings to be cancelled.
Booking	URF 2.18	The reschedule() method of the Booking class allows bookings to be modified.
BookingStatus	URF 2.19	The BookingStatus interface defines booking states such as pending, confirmed, and cancelled, enabling the system to determine whether cancellation rules can be applied to a booking.
Activity	URF 2.20	The liveTrackingData attribute of the Activity class allows real-time activity tracking during a pet care session.
Activity	URF 2.21	The sessionLog attribute of the Activity class stores summaries of completed sessions.
PetMinder	URF 3.1	The serviceDescription, supportedPetTypes, servicePricing, and averageRating attributes within the PetMinder class allow a pet minder to create and manage their professional service profile.
PetMinder	URF 3.2	The supportedPetTypes attribute of the PetMinder class stores the list of pet types that the pet minder can care for.

PetMinder	URF 3.3	The servicePricing attribute in the PetMinder class stores the cost of services offered by the pet minder and can be modified when pricing changes.
PetMinder	URF 3.4	The updateAvailability() method in the PetMinder class allows pet minders to modify their availability schedule so that it can be used during booking searches.
PetMinder	URF 3.5	The association between BookingRequest and PetMinder allows pet minders to retrieve and view incoming booking requests submitted by pet owners.
PetMinder	URF 3.6	The acceptBooking() and declineBooking() methods in the PetMinder class allow a pet minder to respond to a booking request and update the booking's status.
PetMinder	URF 3.7	The cancel() method in the Booking class and the CancellationPolicy interface allows pet minders to cancel confirmed bookings according to system rules.
PetMinder	URF 3.8	The association between Booking, PetProfile, and PetOwner allows pet minders to view booking details including pet information and owner care instructions.
PetMinder	URF 3.9	The sessionStatus attribute in the Activity class represents the lifecycle of a pet minding session and allows a pet minder to start a session when its status changes to STARTED.
PetMinder	URF 3.10	The sessionStatus attribute in the Activity class also allows a session to be terminated by updating its status to COMPLETED.
Activity	URF 3.11	The submitLog() method of the Activity class allows pet minders to upload activity reports
Activity	URF 3.12	The liveTrackingData attribute and updateTracking() operation in the Activity class allow pet minders to provide real-time activity updates during a session.
Administrator	URF 4.1	The viewAllUsers() method of the Administrator class allows administrators to monitor all users.
Administrator	URF 4.2	The suspendUser() method of the Administrator class allows administrators to suspend users
Administrator	URF 4.3	The verifyPetMinder() method of the Administrator class allows administrators to verify service providers.
Administrator	URF 4.4	The resolveDispute() method of the Administrator class allows administrators to manage and resolve booking conflicts.
Administrator	URF 4.5	The moderateReview() method of the Administrator class allows administrators to moderate user written reviews.
Administrator	URF 4.6	The logAction() method of the Administrator class records administrative actions which are stored as entries in the AdminLog class with associated timestamps.
CancellationPolicy	URF 2.17	The canCancel() method of the CancellationPolicy class determines whether a booking can be cancelled according to the system rules.

3. Design Discussion (20%)

1. Design Decisions on Key Attributes and Class Structure

The user class stores a passwordHash rather than a normal plain password. Hashing ensures credentials are irreversible meaning even administrators or developers with legitimate database access cannot retrieve user passwords.

The twoFactorEnabled is an attribute included as Boolean to provide an additional layer of authentication, applying across all Role types.

averageRating is placed on both PetOwner and PetMinder separately. Combining them into a single rating will provide a meaningless and misleading metric, because a PetMinder is rated on service quality, while a PetOwner is rated on their reliability.

medicalInfo and dietaryRequirements are stored directly on PetProfile as simple descriptive fields rather than as separate classes to add specifc requirements or information that minder should be aware of beforehand.

careInstructions is placed on Booking class rather than on PetProfile since careInstructions can vary on each booking depending on the location, duration or situation.

AdminLog stores action, timestamp and adminId to maintain a complete audit trail ensuring that every administrative task can be traced back to the administrator for accountability

cancellationDeadline is stored in each booking rather than on CancellationPolicy since the deadline is specifc to each booking rather than a general rule.

Booking doesn't include a payment system as payment system was considered as outside of the current scope of the system. Including payment attributes such as amount or payment method without a supporting payment class or external service would be incomplete and misleading in the design.

PetProfile does not store an ownerId attribute directly as this relationship is already handled through the composition association with PetOwner. Storing it as both an attribute and an association would create unnecessary data redundancy.

PetMinder/PetOwner does not have a separate location attribute as location is already inherited from User. Duplicating it on PetMinder/PetOwner would violate the principle of avoiding redundancy.

RateAndReview does not store a bookingId as a direct attribute as this relationship is handled through the association with Booking in the diagram, meaning storing it as both an attribute and an association would create unnecessary redundancy.

2. Associations, Compositions and Use Dependencies

Generalisations:

PetOwner and PetMinder all generalise from Role, which in turn generalises from User and and the Administrator is generalised directly from the user. This reflects requirement URF 1.1 which states that a user must be able to register as a Pet Owner, Pet Minder or both, and URF 1.11 which requires users to switch between roles. All role types inherit core account attributes from User such as login and authentication while each adds its own specific behaviour on top.

Compositions:

PetOwner has a composition with PetProfile (1 to 0..). This directly fulfils URF 2.1 which requires a Pet Owner to create one or more pet profiles. Composition is correct here because a pet profile has no purpose without its owner — if the owner account is deleted the profiles should be deleted with it. The 0.. allows an owner to exist without pets initially, which is realistic as a newly registered owner may not have added any pets yet.

Booking has a composition with Activity (1 to 1). This fulfils URF 2.20 and URF 3.11 which require live activity tracking during a session and the ability to submit a session activity log. Since the activity log is meaningless without its associated booking, composition is the correct relationship ensuring the activity cannot exist independently.

Administrator has a composition with AdminLog (1 to 0..*). This directly fulfils URF 4.6 and constraint URC 1.2 which both explicitly require all administrative actions to be logged with accurate timestamps and user identifiers. Composition ensures log entries are permanently tied to and owned by the administrator who performed them.

Associations:

PetOwner associates with BookingRequest (1 to 0..*) meaning one owner can make many booking requests over time. This fulfils URF 2.14 which requires a Pet Owner to be able to request a booking. Each BookingRequest associates with PetMinder (0.. to 1) meaning a minder can receive multiple requests but each request targets exactly one minder, which is consistent with the Make Booking use case where the owner selects a specific minder before submitting a request.

BookingRequest associates with Booking (1 to 1) meaning every booking request results in exactly one booking record. This design cleanly separates the act of requesting a booking from the confirmed booking itself.

PetOwner associates with Booking (1 to 0..*) meaning one owner can have many bookings over time but each booking belongs to exactly one owner. This is consistent with URF 2.14 and the Make Booking use case.

PetMinder associates with Booking (1 to 0..*) meaning a minder can have multiple active or past bookings but each booking is linked to exactly one minder, fulfilling URF 3.5 and URF 3.6.

Booking associates with PetProfile (1 to 1..*) meaning one booking must involve at least one pet but can cover multiple pets simultaneously. This is directly justified by the Make Booking use case which states the owner must select the pets involved.

Booking associates with RateAndReview (1 to 0..2) meaning after a completed booking both parties can leave a review, but neither is required to. The maximum of 2 reflects that only the owner and minder involved can review each other, fulfilling URF 1.12 and URF 1.13, and directly addressing the trust gap identified in our domain analysis where competitor platforms like Rover and Pawshake were noted for their review systems

Realisations and Interface Dependencies:

PetOwner uses SearchService keeping search logic separate from the owner class. This fulfils URF 2.9, URF 2.10 and URF 2.11 which require searching for minders by location, filtering by availability and filtering by rating. Our domain analysis identified that all competing platforms including Rover and Pawshake provide search and discovery as a core feature, confirming this as a fundamental requirement. Using an interface keeps the search logic flexible meaning how searching works can change independently without affecting PetOwner

Booking uses BookingStatus which defines the possible states a booking can be in — pending, confirmed, cancelled and complete. Rather than storing status as a plain String, modelling it as an interface ensures that status behaviour is properly defined and consistently applied across all booking instances. This supports the full booking lifecycle described in both the Make Booking and View Booking Requests use cases.

Booking uses CancellationPolicy which defines whether a booking can be cancelled through the canCancel (booking: Booking): Boolean method. This fulfils URF 2.17 and URF 3.7 which both specify cancellation rules for owners and minders respectively. Using an interface allows different cancellation rules to apply to different booking types without modifying the core Booking class, keeping the design open for extension.

Activity uses SessionStatus enumeration to track whether a session is STARTED, ONGOING or COMPLETED. This supports URF 3.9, URF 3.10 and URF 2.20 which require minders to start and terminate sessions and owners to view live activity tracking. Using an enumeration ensures only valid states can be assigned preventing invalid data entering the system.

RateAndReview uses Administrator through a <<use>> dependency meaning that review moderation is exclusively handled by the Administrator. This ensures that only a system-assigned Administrator has the authority to moderate or remove inappropriate reviews, preventing users from moderating each other's reviews. This directly fulfils URF 4.5 which requires administrators to moderate and remove inappropriate reviews and works alongside the isModerated : Boolean attribute on RateAndReview which flags whether a review has been actioned by an Administrator.

3. Differences Between the Domain Model and Class Diagram

Several changes were made when transitioning from the domain model to the class diagram, each justified by design improvements and a deeper analysis of the system requirements.

The most significant addition to the class diagram that did not exist in the domain model is the Role class. In the domain model PetOwner, PetMinder and Admin were treated as separate entities with no shared structure. The Role class was introduced in the class diagram because requirement URF 1.11 explicitly states that users must be able to switch between Pet Owner and Pet Minder roles, which is impossible to achieve cleanly without a role abstraction. User acts as the central account holder, directly generalising Administrator while Role acts as the bridge, directly implementing the Player-Role pattern.

Admin in the domain model was a standalone entity. In the class diagram it has been formalised as Administrator with a direct generalisation to User rather than through Role. This is a deliberate design decision — unlike PetOwner and PetMinder, an administrator is system-assigned rather than user-selected, so connecting it directly to User ensures administrators cannot be assigned through the switchRole() mechanism, which is an important security consideration.

Profile existed as a separate entity in the domain model connected to User. In the class diagram this has been replaced by PetProfile which is specifically owned by PetOwner through a composition. This change reflects a more accurate understanding of the domain — a profile in this system is always a pet profile belonging to an owner, not a generic user profile.

SearchResult was present in the domain model as a separate entity. This was removed in the class diagram as it represents a system function rather than a real domain entity. It has been replaced by the SearchService interface on PetOwner, fulfilling URF 2.9, URF 2.10 and URF 2.11 in a cleaner and more appropriate way.

CareInstruction was modelled as a separate class in the domain model connected to Booking. In the class diagram this has been simplified to a careInstructions : String attribute directly on Booking. Since care instructions do not require their own behaviour or associations, promoting them to a full class added unnecessary complexity. This also better reflects URF 2.16 which simply requires the owner to attach care instructions to a booking rather than managing them as independent entities.

Availability existed as a standalone class in the domain model connected to PetMinder. In the class diagram this has been absorbed into PetMinder through the updateAvailability() method and supportedPetTypes attribute. Since availability is a property of a minder rather than an independent entity, this simplification produces a cleaner and more cohesive design while still fulfilling URF 3.4 which requires minders to set and update their availability.

Activity and RateAndReview both existed in the domain model and have been retained in the class diagram. However, they have been significantly enhanced with proper attributes, methods and associations. Activity now includes liveTrackingData, sessionLog and sessionStatus to fulfil URF 2.20, URF 3.11 and URF 3.12. RateAndReview now includes rating, comment and isModerated to fulfil URF 1.12, URF 1.13 and URF 4.5.

Three entirely new classes were introduced that had no equivalent in the domain model. BookingRequest was added to properly separate the request stage from a confirmed

booking, reflecting the full lifecycle described in the View Booking Requests use case. AdminLog was added to fulfil URF 4.6 and URC 1.2 which explicitly require all administrative actions to be logged with timestamps — this was overlooked in the initial domain model. SessionStatus was added as an enumeration to provide a type-safe way to track session states, replacing what would have been an error-prone plain String field.

4. Applied Design Patterns and Justification

The Player-Role Pattern (also known as Bridge):

The primary design pattern applied is the Player-Role pattern, also referred to as the Bridge pattern. The context for this pattern is when objects change roles or have different roles based on context — precisely the situation here as a person may own pets and also provide minding services, directly reflecting requirement URF 1.1 which states a user must be able to register as a Pet Owner, Pet Minder or both.

The antipattern would have been making PetOwner and PetMinder direct subclasses of User. As this permanently binds a user to a single role making role switching impossible, resulting in data duplication and poor user experience, and would directly violate URF 1.11 which requires users to switch between roles.

Instead, Role acts as an intermediary. In terms of the pattern, User is the Player, Role is the AbstractRole and PetOwner and PetMinder are the concrete role implementations. Role is designed as an abstract class meaning it cannot be instantiated directly. This is correct because in the real world a user is never just a 'Role' — they are always specifically an owner or minder. The switchRole() method on User enables dynamic role switching at runtime without creating a new account, which is exactly the problem this pattern solves.

Administrator is intentionally kept outside the Role hierarchy by connecting directly to User. This ensures administrators cannot be assigned through switchRole() and must be system-assigned, which is an important security consideration ensuring regular users cannot escalate their own privileges.

Use Dependencies (<<use>>) and Interface Design:

SearchService, BookingStatus and CancellationPolicy are all modelled as interfaces used through <<use>> dependencies. In simple terms an interface defines what something must be able to do without specifying exactly how. This keeps the design flexible and extensible — CancellationPolicy allows different cancellation rules for different booking types without modifying Booking, BookingStatus ensures consistent state management across all booking instances, and SearchService keeps search logic separate from PetOwner so it can change independently. This approach aligns with the open/closed principle — the system is open for extension but closed for modification.

SessionStatus Enumeration:

SessionStatus is modelled as an enumeration ensuring only valid session states (STARTED, ONGOING, COMPLETED) can be assigned to an Activity. This prevents invalid data from entering the system and improves reliability, supporting URF 3.9 and URF 3.10 which require

minders to start and terminate sessions. Using an enumeration rather than a plain String is a deliberate design decision that enforces data integrity at the model level.

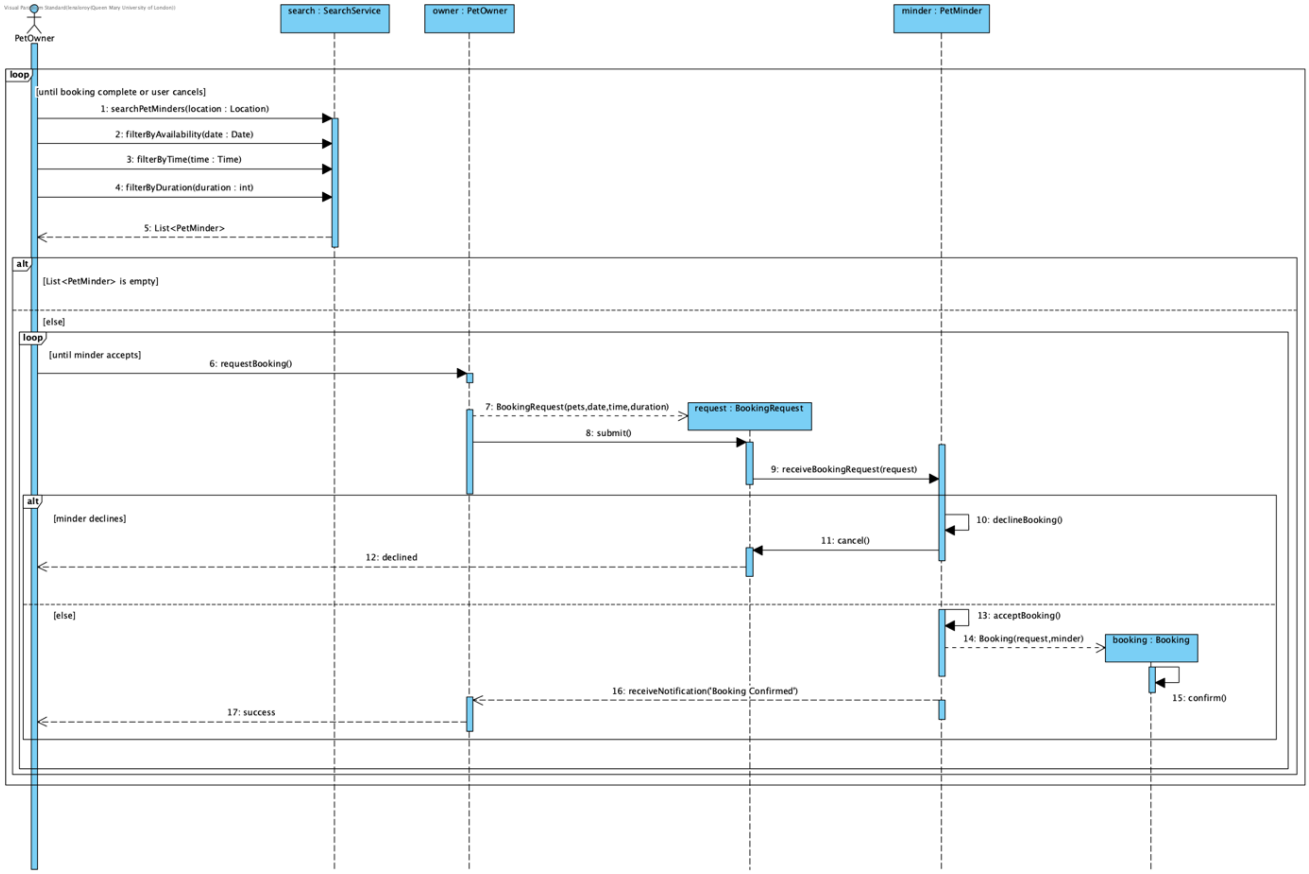
4. Sequence Diagrams (30% => 3x10%)

Sequence Diagram: Make Booking

The pet owner can search for a pet minder that provides a walking services within a set location radius and can schedule if for a date and time of their convenience

Pre-Requisites

- The pet owner is logged in
- The pet owner has set up pet profiles for all the pets being walked
- The pet owner has location services enabled

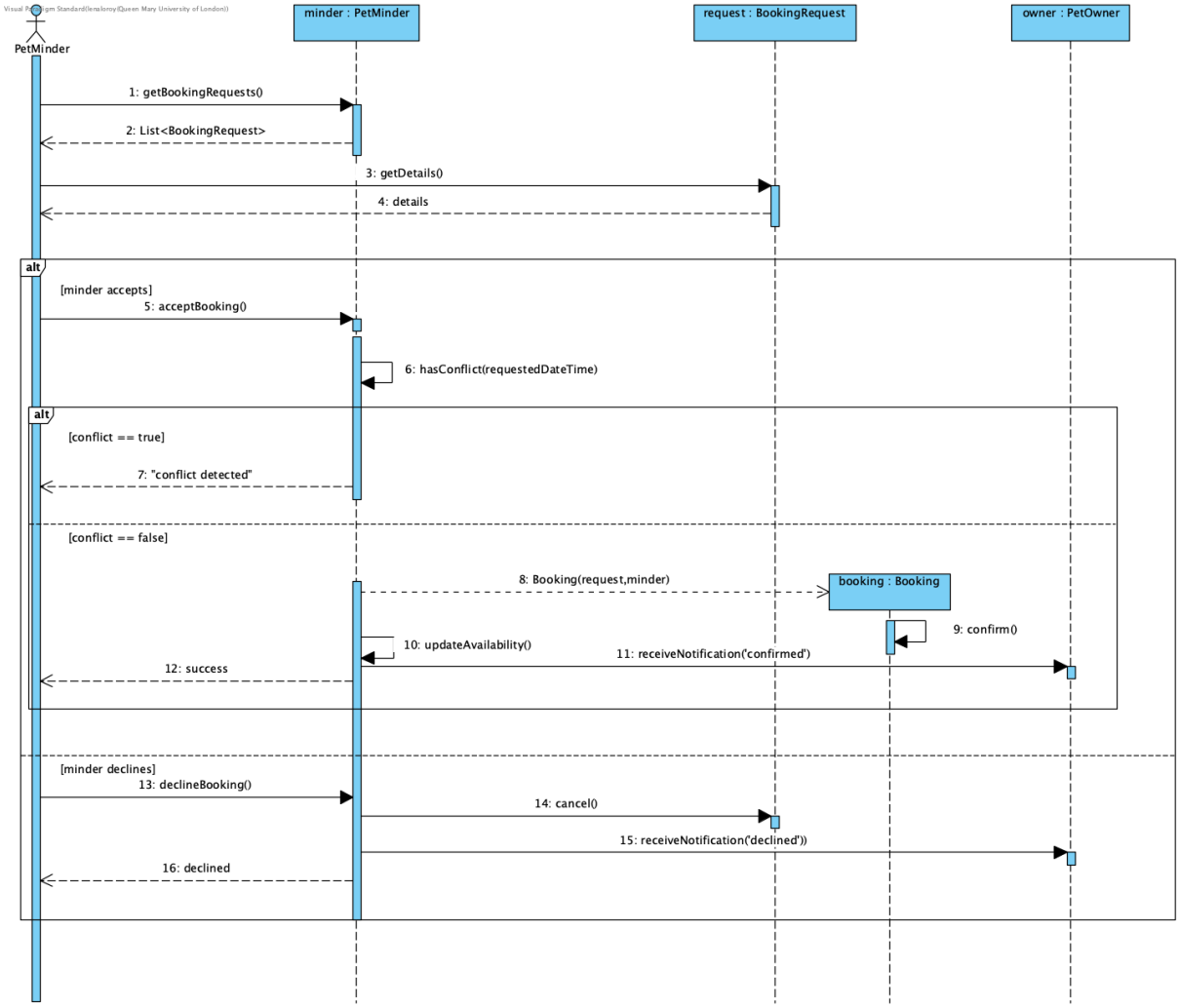


Sequence Diagram: View Booking Requests

A pet minder can view incoming booking requests from Pet Owners, review the details of the pets and the requested time slot, and choose to accept or decline the request. The system updates availability and notifies the pet owner accordingly.

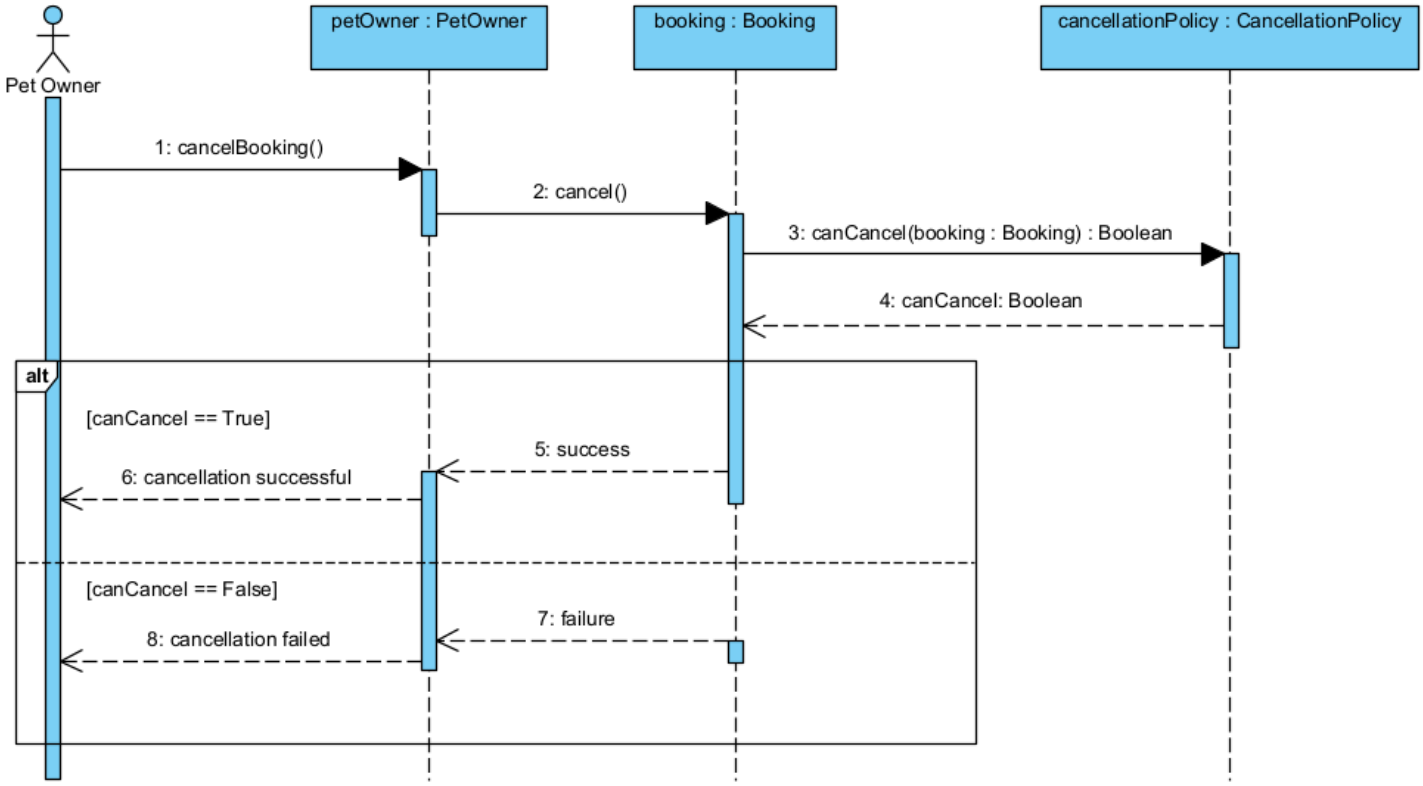
Pre-Requisites

- The pet minder is logged in.
- The pet minder has an active profile with availability set.
- A booking request has been submitted by a pet owner



Sequence Diagram: Cancel Booking Use Case

This sequence diagram models the Cancel Booking use case for our system. A Pet Owner may decide to cancel an existing booking with a Pet Minder. The system must check whether the booking is still eligible for cancellation. If the deadline has not passed, the booking is cancelled successfully; otherwise, the system informs the user that cancellation is not allowed.

Pre-Requisites

1. The Pet Owner is already logged into the system.
2. A valid Booking object exists in the system.
3. A CancellationPolicy instance is associated with the booking.
4. The Pet Owner has navigated to the booking they wish to cancel.